

1. SQL Basics – Interview Questions

Theoretical Questions

1. What is SQL?

Answer:

SQL (Structured Query Language) is used to communicate with databases to store, retrieve, update, and delete data.

2. What are types of SQL commands?

Answer:

- **DDL (Data Definition Language):** CREATE, ALTER, DROP, TRUNCATE
 - **DML (Data Manipulation Language):** INSERT, UPDATE, DELETE
 - **DCL (Data Control Language):** GRANT, REVOKE
 - **TCL (Transaction Control Language):** COMMIT, ROLLBACK, SAVEPOINT
-

3. Difference between DBMS and RDBMS?

Feature	DBMS	RDBMS
Structure	Non-relational	Relational (tables)
Relationships	Not supported	Supported
Example	File system	MySQL, Oracle

4. What is a table?

A collection of data organized in rows and columns.

5. What is a row and column?

- **Row:** A single record
 - **Column:** Attribute/field of data
-

6. What are data types?

Define the type of data a column can hold.

Examples:

- INT → numbers
 - VARCHAR → text
 - DATE → date
 - FLOAT → decimal numbers
-

7. Difference between DELETE, TRUNCATE, DROP?

Command	Description
DELETE	Removes specific rows
TRUNCATE	Removes all rows (fast)
DROP	Deletes entire table

8. What is PRIMARY KEY?

A column that uniquely identifies each record.

9. What is NULL?

Represents missing or unknown data.

10. What is DEFAULT constraint?

Provides a default value if none is specified.

11. What is a constraint?

Rules applied on columns to maintain data accuracy and integrity.

12. Types of constraints?

- NOT NULL
 - UNIQUE
 - PRIMARY KEY
 - FOREIGN KEY
 - CHECK
 - DEFAULT
-

13. What is normalization?

Process of organizing data to reduce redundancy.

14. Types of normalization?

- 1NF → Atomic values
- 2NF → Remove partial dependency
- 3NF → Remove transitive dependency

Commands & Behavior

15. What is the difference between DROP and TRUNCATE?

- DROP → removes table completely
 - TRUNCATE → removes all data but keeps structure
-

16. Is TRUNCATE faster than DELETE? Why?

Yes, because it does not log individual row deletions.

17. Can we rollback TRUNCATE?

No (in most DBMS), because it is auto-commit.

18. What is auto-commit?

Every SQL statement is automatically committed without manual control.

19. Difference between DELETE and TRUNCATE?

DELETE	TRUNCATE
Row-wise deletion	Removes all rows
Can use WHERE	No WHERE
Slower	Faster

20. What is a schema?

Logical structure of a database (tables, views, etc.).

21. What is metadata?

Data about data (e.g., table structure).

22. What is a view?

A virtual table created from a query.

23. Difference between table and view?

Table	View
Stores data	Virtual
Physical storage	No storage

24. What is indexing?

Technique to improve query performance.

Data Types Deep Understanding

25. Difference between INT and FLOAT?

- INT → whole numbers
 - FLOAT → decimal numbers
-

26. What is VARCHAR?

Variable-length string data type.

27. Difference between CHAR and VARCHAR?

CHAR	VARCHAR
Fixed size	Variable size
Faster	Flexible

28. What is DATE data type?

Stores date values (YYYY-MM-DD).

29. What is NULL vs 0?

- NULL → no value
 - 0 → actual numeric value
-

Advanced Basic Concepts

30. What is a composite key?

Combination of multiple columns as a primary key.

31. Can NULL be part of PRIMARY KEY?

No.

32. Can NULL be part of UNIQUE key?

Yes (depends on DBMS).

33. What is candidate key?

All possible keys that can uniquely identify rows.

34. What is alternate key?

Candidate keys not chosen as primary key.

35. What is surrogate key?

Artificial key (like auto-increment ID).

36. What is natural key?

Real-world data used as a key (e.g., email).

37. What is denormalization?

Combining tables to improve performance.

38. What is redundancy?

Duplicate data in a database.

39. What is data integrity?

Accuracy and consistency of data.

Transactions & Control

40. What is a transaction?

A group of SQL operations executed together.

41. What are ACID properties?

- **A** → Atomicity
 - **C** → Consistency
 - **I** → Isolation
 - **D** → Durability
-

42. What is COMMIT?

Saves changes permanently.

43. What is ROLLBACK?

Undo changes.

44. What is SAVEPOINT?

Temporary point to rollback partially.

Interview Tricky Questions

45. What happens if you insert duplicate PRIMARY KEY?

Error occurs.

46. Can a table exist without a primary key?

Yes, but not recommended.

47. Can we have multiple UNIQUE constraints?

Yes.

48. What is default value behavior?

If no value is provided, default is inserted.

49. What is difference between SQL and MySQL?

- SQL → language
 - MySQL → database software
-

50. What is case sensitivity in SQL?

Depends on DBMS (MySQL is case-insensitive for keywords).

51. What is the use of ALTER?

Modify table structure.

52. What is data definition?

Structure of database objects.

53. What is data manipulation?

Operations on data (insert/update/delete).

54. Why SQL is important for Data Analysts?

Because it helps to:

- Extract data
- Analyze datasets
- Generate reports

Practical Questions (With Queries)

1. Create a table

Question: Create a table `employees`

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary FLOAT,  
    joining_date DATE  
);
```

2. Insert data

```
INSERT INTO employees (id, name, salary, joining_date)  
VALUES (1, 'Adarsh', 50000, '2024-01-10');
```

3. Insert multiple rows

```
INSERT INTO employees VALUES  
(2, 'Rahul', 60000, '2023-05-20'),  
(3, 'Amit', 55000, '2022-03-15');
```

4. Select data

```
SELECT * FROM employees;
```

5. Add a new column

```
ALTER TABLE employees  
ADD department VARCHAR(30);
```

6. Modify column

```
ALTER TABLE employees  
MODIFY salary DOUBLE;
```

7. Rename column

```
ALTER TABLE employees  
RENAME COLUMN name TO emp_name;
```

8. Delete specific rows

```
DELETE FROM employees  
WHERE id = 2;
```

9. Remove all data

```
TRUNCATE TABLE employees;
```

10. Drop table

```
DROP TABLE employees;
```

Important Interview Scenarios

1. Create table with constraints

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    age INT CHECK (age >= 18),
```

```
city VARCHAR(30) DEFAULT 'Delhi'
);
```

2. Insert with NULL

```
INSERT INTO employees (id, name, salary)
VALUES (4, 'Karan', NULL);
```

3. What happens if PRIMARY KEY is duplicated?

It will throw an error (violates uniqueness).

4. Can a table have multiple primary keys?

No, but it can have a **composite primary key**.

5. Difference between VARCHAR and CHAR?

- VARCHAR → variable length
 - CHAR → fixed length
-

Theoretical Scenario-Based SQL Questions

1. Data Deletion Strategy

Scenario:

You need to remove all data from a table quickly, and you don't need rollback.

Question: Which command will you use and why?

Answer:

Use `TRUNCATE` because it is faster and removes all rows without logging individual deletions.

2. Data Safety Scenario

Scenario:

You accidentally deleted some records and want to recover them.

Question: Which command could have helped?

Answer:

`ROLLBACK` (if transaction was not committed).

3. Duplicate Data Issue

Scenario:

Your table has duplicate records.

Question: How will you prevent duplicates in future?

Answer:

Apply **UNIQUE constraint** or **PRIMARY KEY**.

4. Missing Data Problem

Scenario:

Some columns have missing values.

Question: How do you handle it at database level?

Answer:

- Use `DEFAULT` values
 - Use `NOT NULL` constraint
 - Handle using functions like `COALESCE`
-

5. Data Integrity Scenario

Scenario:

You want to ensure that every employee must belong to a valid department.

Question: What will you use?

Answer:

Foreign Key constraint to maintain referential integrity.

6. Performance Issue

Scenario:

Your query is slow when searching by salary.

Question: What can improve performance?

Answer:

Create an **INDEX** on the salary column.

7. Table Design Scenario

Scenario:

You are designing a student table.

Question: How will you ensure each student is uniquely identified?

Answer:

Use a **PRIMARY KEY**.

8. Multiple Column Uniqueness

Scenario:

A combination of two columns must be unique.

Answer:

Use **COMPOSITE PRIMARY KEY** or **UNIQUE (col1, col2)**

9. Data Consistency Scenario

Scenario:

Multiple users are updating the same data.

Question: How to ensure consistency?

Answer:

Use **transactions** + **ACID properties**

10. Rollback Partial Changes

Scenario:

You want to undo part of a transaction.

Answer:

Use **SAVEPOINT**

Intermediate Theoretical Scenarios

11. Data Type Choice

Scenario:

You need to store phone numbers.

Question: Which data type and why?

Answer:

Use `VARCHAR` (not `INT`) because:

- May contain leading zeros
 - Not used for calculations
-

12. Fixed vs Variable Data

Scenario:

You store country codes like “IN”, “US”.

Answer:

Use `CHAR(2)` (fixed length)

13. Table Removal Decision

Scenario:

You want to completely remove table structure.

Answer:

Use `DROP`

14. Partial Data Removal

Scenario:

You want to delete only some records.

Answer:

Use `DELETE` with `WHERE`

15. Default Value Scenario

Scenario:

If no city is provided, default should be “Delhi”.

Answer:

Use `DEFAULT 'Delhi'`

Advanced Theoretical Scenarios

16. Normalization Need

Scenario:

Your table has repeated data.

Question: What should you do?

Answer:

Apply **Normalization** to reduce redundancy.

17. Denormalization Case

Scenario:

You need faster reporting queries.

Answer:

Use **Denormalization** (combine tables)

18. Null Handling Decision

Scenario:

You must ensure salary is always present.

Answer:

Use `NOT NULL`

19. Key Selection

Scenario:

Email is unique but may change.

Question: Should it be primary key?

Answer:

No → use **surrogate key (ID)**

20. Data Validation

Scenario:

Age must be greater than 18.

Answer:

Use `CHECK (age > 18)`

Tricky Interview Scenarios

21. Can a table have no primary key?

Answer: Yes, but not recommended.

22. Can NULL be compared using = ?

Answer: No → use `IS NULL`

23. Why is TRUNCATE faster than DELETE?

Answer:

Because it doesn't log row-by-row deletion.

24. Can we use WHERE in TRUNCATE?

Answer: No

25. What happens if foreign key is violated?

Answer:

Database will throw an error.

Query Scenario-Based SQL Questions

1. Employee Salary Update Scenario

Question:

You need to increase salary by 10% for employees in the IT department.

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'IT';
```

Tests: UPDATE + WHERE

2. Missing Data Scenario

Question:

Find employees whose salary is not provided.

```
SELECT *
FROM employees
WHERE salary IS NULL;
```

Tests: NULL handling

3. Duplicate Data Scenario

Question:

Find duplicate emails in a users table.

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Tests: GROUP BY + HAVING

4. Data Cleaning Scenario

Question:

Replace NULL salary with 0.

```
SELECT name, COALESCE(salary, 0) AS salary
FROM employees;
```

Tests: COALESCE

5. Table Modification Scenario

Question:

Add a column `bonus` to `employees`.

```
ALTER TABLE employees
ADD bonus INT;
```

Tests: ALTER

6. Business Insight Scenario

Question:

Find total salary expense per department.

```
SELECT department, SUM(salary)
FROM employees
GROUP BY department;
```

Tests: Aggregation

7. Filtering Groups Scenario

Question:

Find departments with more than 5 employees.

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Tests: HAVING

8. Relationship Scenario

Question:

Get employee names along with their manager names.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

Tests: Self JOIN

9. Data Deletion Scenario

Question:

Delete employees who left the company (status = 'inactive').

```
DELETE FROM employees
WHERE status = 'inactive';
```

Tests: DELETE

10. Table Reset Scenario

Question:

Remove all data but keep table structure.

```
TRUNCATE TABLE employees;
```

Tests: TRUNCATE

11. Top Performer Scenario

Question:

Find highest paid employee.

```
SELECT *  
FROM employees  
ORDER BY salary DESC  
LIMIT 1;
```

Tests: ORDER BY + LIMIT

12. Second Highest Salary Scenario

```
SELECT MAX(salary)  
FROM employees  
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Very frequently asked

13. Hiring Scenario

Question:

Insert a new employee record.

```
INSERT INTO employees (id, name, salary)  
VALUES (10, 'Ravi', 45000);
```

14. Data Validation Scenario

Question:

Ensure age must be ≥ 18 while creating table.

```
CREATE TABLE users (  
    id INT,  
    age INT CHECK (age >= 18)  
);
```

15. Customer Analysis Scenario

Question:

Find customers who placed more than 3 orders.

```
SELECT customer_id, COUNT(*)  
FROM orders
```

```
GROUP BY customer_id  
HAVING COUNT(*) > 3;
```

16. Revenue Insight Scenario

Question:

Find total revenue.

```
SELECT SUM(amount) FROM orders;
```

17. Orphan Records Scenario

Question:

Find employees without department.

```
SELECT *  
FROM employees  
WHERE dept_id IS NULL;
```

18. Join Missing Data Scenario

Question:

Find employees whose department does not exist.

```
SELECT e.*  
FROM employees e  
LEFT JOIN departments d  
ON e.dept_id = d.dept_id  
WHERE d.dept_id IS NULL;
```

19. Performance Scenario

Question:

Improve query performance on salary column.

```
CREATE INDEX idx_salary ON employees(salary);
```

20. Real-Time Reporting Scenario

Question:

Generate monthly sales report.

```
SELECT MONTH(order_date), SUM(amount)
FROM orders
GROUP BY MONTH(order_date);
```

Advanced Scenarios (Very Important)

21. Top 3 Employees per Department

```
SELECT *
FROM (
    SELECT *, RANK() OVER (PARTITION BY department ORDER BY salary DESC) rnk
    FROM employees
) t
WHERE rnk <= 3;
```

22. Running Total

```
SELECT order_date,
SUM(amount) OVER (ORDER BY order_date)
FROM orders;
```

23. Salary Above Department Average

```
SELECT name, salary, department
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e1.department = e2.department
);
```

24. Latest Order per Customer

```
SELECT *
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date
DESC) rn
    FROM orders
```

```
) t  
WHERE rn = 1;
```

25. Percentage Contribution

```
SELECT department,  
SUM(salary) * 100.0 / (SELECT SUM(salary) FROM employees)  
FROM employees  
GROUP BY department;
```